Least-Privilege Data Access for LLM Database Agents: Task-Scoped Capabilities and Session-Level Inference Control

Sanjay Mishra

Abstract—Agents built on large language models now query enterprise databases through tool interfaces under one broad database account, so a narrow task can read any column the account allows, and a run of individually allowed aggregates can reveal one person's value. We place a gateway in front of the agent's `run_sql` tool with two controls. A task-scoped capability, derived from the task text by an LLM and closed under join keys, states which columns a task may filter on, aggregate or return; identifying columns always need a human. A session audit computes the entities behind every answer that depends on a protected column and refuses a query whose groups, nested differences or protected-column filters rest on fewer than k entities, or which would make one entity's exact value a linear combination of earlier answers. On 220 BIRD tasks, derived capabilities cut the columns per task from 72.5 to about 7.5 and changed execution accuracy by -5.0 points with gpt-4o-mini and by +0.5 with Claude Sonnet 5.5. A plausible injected instruction widened most capabilities to identifying columns; the approval rule stopped every such grant at the cost of sending 15.0% of clean tasks to a human. On length-matched sessions the audit prevented every two- and three-query differencing attack while refusing 3.3% of benign queries, and no adaptive LLM attacker obtained an exact target value. As an MCP proxy on PostgreSQL at TPC-H scale factor 1, the gateway reproduced its library decisions on 2,942 queries with a median check of 1.4 ms. What bounds disclosure is state about answers, not about dialogue.

Index Terms—Database access control, inference control, query auditing, large language model agents, text-to-SQL, least privilege.

1 Introduction

An analyst asks an assistant for revenue by region last quarter. The assistant, an agent built on a large language model (LLM), writes SQL, sends it to a database tool over the Model Context Protocol (MCP), and summarizes the rows that come back. The tool holds one database account, typically a reporting role that can read most of the warehouse. The task needed four columns; the agent could read seventy. If the agent is steered by text it reads, such as an instruction hidden in a retrieved document [1], [2], the role and not the task bounds what it can take.

Breadth is one gap. Accumulation is the other. Databases decide each query on its own, but disclosure can come from a combination: two averages whose groups differ by one customer reveal that customer's value, although each average covers many people. Statistical databases called this the tracker problem [3] and built audits for it [4], [5]. Those audits assumed a static query interface. They were never run against an agent that reads each refusal and tries again. Current security work on LLM data access looks at one query at a time: role-aware text-to-SQL benchmarks [6], [7] check whether a model respects a fixed role, attacks extract schemas or data through generated SQL [8], [9], and agent authorization frameworks scope which tools a task may call [10], [11]. Multi-turn jailbreaks such as Crescendo [12] showed that harmless-looking steps can add up to a harmful result, and sequential monitors [13] watch the dialogue for that build-up. None of this limits what an allowed SQL tool returns for a given task, or tracks what a session's answers have already released. Our earlier conversation-level risk scores [14] did not separate risk from session length once that shortcut was controlled; this paper moves the state from the conversation to the data that left the database.

This paper makes the following contributions:

1) **Mode-aware task capabilities.** A capability lists the columns a task may use and whether it may filter on them, aggregate them or return them; it is derived from the task text, closed under join keys and widened only by escalation, with identifying columns reserved for a human (Section 5).
2) **A privilege–accuracy measurement on BIRD.** Over 220 tasks and four LLMs, one open-weight, we report how many columns derived capabilities remove and what they cost in execution accuracy against the role and against the gold query's own footprint (Section 7.1).
3) **An injection study of capability derivation.** Blunt, plausible and hint-borne instructions widen derived capabilities by different amounts, and a deterministic approval ceiling stops identifying grants at a measured human cost (Section 7.2).
4) **An answer-level session audit.** Small-cell, nested-pair, predicate-dependence and incremental linear tests refuse only the query that would complete a disclosure, with a proof for exact sums and a strict variant whose refusals carry no information about

• *S. Mishra is an independent researcher, Raleigh, NC, USA. E-mail: sanmish4@icloud.com*

Code, benchmark generators and results: https://github.com/sanmish4ds/least-privilege-llm-db-agents.

protected values (Section 5.2).

5) **Evaluation against scripted and adaptive attackers.** A session benchmark with matched lengths, LLM attackers that read refusals, an algebraic disclosure oracle, a differential-privacy baseline and human-written BIRD workloads measure prevention and benign cost (Sections 7.3–7.6).
6) **A deployable MCP gateway.** The controls run either on a read-only connection or as a proxy in front of an existing database MCP server, and were replayed end to end on PostgreSQL at TPC-H scale factor 1 (Section 7.7); code, generators and result files regenerate every number in this paper.

Scope: This is not a text-to-SQL leaderboard, and we train no model. The gateway does not detect prompt injection; it limits what an injected agent can obtain. It answers exactly or refuses, so it is not a differential-privacy system, although we compare with one. It assumes the gateway and the database account are not themselves compromised.

2 Related Work

2.1 Access control and purpose in database systems

Role-based access control assigns permissions to roles rather than users [15]; fine-grained access control restricts rows and columns by rewriting queries against authorization views [16]. Hippocratic databases made the purpose of each access a first-class concept, with purpose specification and limited disclosure [17], and purpose-based access control formalized intended and access purposes in relational systems [18]. Anomaly detectors learn per-role profiles of SQL activity and flag deviations [19]. Purposes and roles in this line are declared by people and fixed for long periods; none derives a scope per task from text, and none is evaluated against a program that chooses its own queries.

2.2 Inference control and query auditing

Answering only large-group aggregates is not enough: a tracker combines allowed queries to isolate an individual [3]. The audit expert decides by linear algebra whether a set of answered sum queries determines an individual value [4]. Enough noise-free aggregate answers allow reconstruction of a database [20], and refusals themselves can leak unless the audit decision is simulatable from public information [5]. Differential privacy replaces exact answers with noisy ones [21] and has been engineered for SQL [22]; k-anonymity protects released tables rather than query streams [23]. These methods target static interfaces and human analysts. Their cost on agent workloads, and their behavior against agents that read refusals and adapt, had not been measured, and differential privacy does not fit tasks that need exact values.

2.3 Security of LLM data agents and text-to-SQL

Text-to-SQL is benchmarked on Spider [24] and BIRD [25]. Prompts can be turned into SQL injections [2], schemas can be inferred from generated SQL without access [8], data agents with planning and session state open new attack paths [9], and instructions hidden in retrieved content can redirect an application [1]. Benchmarks now test role-based access control in text-to-SQL [6] and whether LLMs reason correctly about role-conditioned access [7]. These evaluations judge one query against a fixed role, and the defense, when there is one, is the model's own compliance.

TABLE 1
Positioning. *Task*: scope derived per task. *Mode*: distinguishes filtering, aggregation and returned values. *Session*: decisions use earlier answers. *Adaptive*: evaluated against an attacker that sees refusals.

Line of work	Task	Mode	Session	Adaptive
RBAC / FGAC [15], [16]	–	partial	–	–
Purpose AC [17], [18]	declared	–	–	–
Auditing [4], [5]	–	aggr.	yes	–
RBAC text-to-SQL [6]	–	–	–	–
Agent authz [10], [11]	tools	–	–	–
Seq. monitors [13]	–	–	dialogue	yes
This work	columns	yes	answers	yes

2.4 Multi-turn attacks and agent authorization

Crescendo [12] is a multi-turn jailbreak that never states its goal. It opens with an innocent question on the topic, then asks the model to build on its own previous replies, a little further each turn, until the model produces what it would have refused if asked directly. The authors compare it to the foot-in-the-door tactic: agreeing to a small request makes agreeing to the next one easier. Because each prompt is benign, filters that judge one prompt at a time miss it, and lightweight monitors that accumulate evidence across a decomposed request do better [13]. Authorization architectures for tool-using agents scope tool access per task [10], and flow-centric policy languages constrain how data moves between agent components [11]. Scope there is set at the granularity of tools, not columns and aggregation modes, and session state describes the dialogue, not the information the database has released. Section 3.1 shows why that difference matters for SQL.

2.5 Positioning

Three lines of work are closest. Against role-aware text-to-SQL [6], we bound the data a task may reach instead of testing whether the model obeys a role. Against agent authorization [10], [11], we scope columns and modes rather than tools. Against classic auditing [4], [5], we keep the algebra but derive scopes per task, audit predicates as well as sums, and evaluate against adaptive LLM attackers. Table 1 summarizes the dimensions.

3 Problem Formulation

3.1 How a session leaks: an example

Fig. 1 shows the pattern in plain form. The agent asks for an average balance over a market segment in one nation, then the same average over the customers with keys below 1,400. Both are ordinary analyst questions, both cover a dozen customers, and a database that checks each query against the role answers both. Subtracting the two totals gives the balance of the one customer who is in the first group and not the second. The lower panel needs no arithmetic at all: a count that is 0 or 1 says whether one customer's balance is

(a) Differencing (tracker)

A: What is the average balance of BUILDING customers in nation 7?

DB: 4,512.30, over 12 customers.

A: And for the same customers with key below 1,400?

DB: 4,301.10, over 11 customers.

A: $12\times4{,}512.30-11\times4{,}301.10 = 6{,}835.50$: customer 1,412's balance.

(b) Threshold bisection

A: How many customers with key 570 have a balance above 5,000?

DB: 1.

A: ... above 7,500? ... above 8,750?

DB: 1. 0.

A: (18 more halvings) The balance is known to the cent.

Fig. 1. Two ways a session leaks one customer's balance through questions that are each allowed (illustrative values; A: agent, DB: database answer). No single query returns the balance, and no query text names the goal.

above a threshold, and halving the interval each time pins the balance to a cent in about 21 questions.

The pattern resembles Crescendo [12]: each step is harmless, and the next step builds on what the previous answers made possible. The difference is where the build-up lives. In Crescendo it is in the dialogue, so a monitor reading the conversation can in principle see it [13]. In Fig. 1 the text of every step is innocuous; the disclosure exists only in the arithmetic relation between answers. A defense therefore has to keep the answers, and specifically the sets of entities behind them. Section 7.3 confirms this empirically: session alarms that score the shape of a session miss differencing attacks almost entirely, while an audit over entity sets stops all of them.

3.2 Threat model

The adversary controls the agent's inputs (the task text, content the agent retrieves) or the agent itself. It may issue any number of queries in a session, read every answer and refusal, and adapt. It cannot change the gateway or the database account, and it cannot open unlimited sessions under one identity, because budgets are keyed to an identity the deploying application authenticates. We protect two properties: (P1) a task cannot use a column beyond its capability without an approved escalation; (P2) within a session, no admitted answer that depends on a protected column rests on fewer than k entities as a group, as a nested difference or through a protected-column filter, and no set of admitted sums determines one entity's protected value exactly. Because escalation of non-identifying columns is automatic, P1 limits an adversarial agent only for identifying columns; for other columns it limits the damage of an honest but mistaken agent. P2 holds against the adversary in full.

3.3 Notation

Table 2 lists the symbols. A capability C assigns each column of $\mathcal{A}$ one of four ordered modes. A column p is *protected* if per-entity values of p may only be aggregated; $E(p)$ is its entity table with key κ. For a query q, $\mathcal{S}(q)$ are the entity sets of its output groups when q aggregates p, and $\mathcal{C}(q)$ are the candidate sets when q filters on p (Section 5.2).

TABLE 2
Notation.

Symbol	Meaning
$\mathcal{A}$	Columns $t.c$ of the database
C	Capability $\mathcal{A} \to \{\bot, \mathsf{filter}, \mathsf{agg}, \mathsf{raw}\}$, $\bot < \mathsf{filter} < \mathsf{agg} < \mathsf{raw}$
$\mu(c, q)$	Mode in which query q uses column c (Eq. 1)
$K(T)$	Primary, foreign and shared-name keys of table set T
P, $E(p)$	Protected columns; entity table and key of p
$\mathcal{S}(q)$, $\mathcal{C}(q)$	Entity sets of q's groups; candidate sets of q's protected filter
$\mathcal{H}$, $\mathcal{H}_\downarrow$	Sets of earlier admitted aggregates and protected filters
k	Minimum number of entities an answer may rest on
$\mathbf{1}_S$, Q	Indicator vector of set S; orthonormal basis of answered indicators
EX	Execution accuracy: result set equals the gold result set

Success and counts: A task *fits* a capability if its gold SQL is admitted under it. An attack session is *prevented* if the gateway refuses its disclosing query or an earlier one. A benign query is a *false refusal* if the gateway refuses it; we report the share of benign queries and of benign sessions with at least one. In the adaptive study a target is *disclosed* if the answers the gateway returned determine its value exactly, as decided by an oracle (Section 6), regardless of what the attacker claims.

4 System Architecture

4.1 Gateway and MCP interface

Fig. 2 shows the gateway. Agents reach databases only through it. It is an MCP server with two tools, `begin_session(purpose)` and `run_sql(sql)`. It executes admitted queries either itself, on a connection whose role has only SELECT and a read-only transaction default, or by forwarding them to a downstream database MCP server, so it can sit in front of an existing vendor server (Section 7.7). A catalog supplies keys, entity tables and per-column sensitivity tags.

4.2 Session semantics

A session starts with a purpose, which names the task and the identity it runs under. The gateway issues the capability once per session and keeps three pieces of state for it: the capability, which only grows through escalation; the ledger of entity sets behind admitted answers, per protected column; and the set of entities whose sensitive columns were returned raw, checked against the purpose's entity budget. A refused query changes none of them, because nothing was released.

4.3 Query pipeline

Algorithm 1 gives the per-query check. The gateway parses the SQL, resolves every column to $t.c$, checks the capability, runs the audit, and only then executes the query; every decision is logged with the task, the capability and any approval. Sections 5.1 and 5.2 define the two checks.

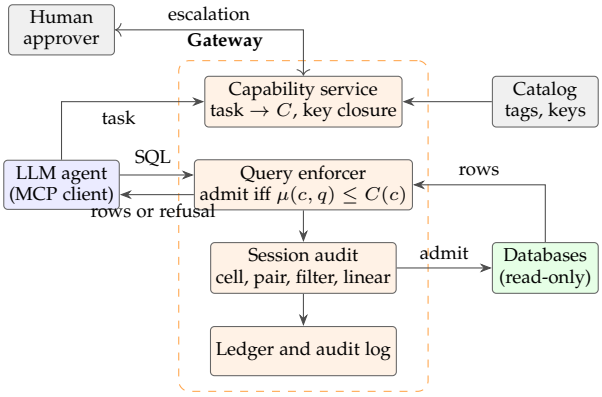


Fig. 2. Gateway between LLM agents and databases. The capability service turns a task into a column-level capability using catalog keys; the enforcer refuses queries that use columns beyond it; the session audit refuses queries whose answer would rest on fewer than k entities given earlier answers; only admitted queries reach the read-only connection. Identifying columns are granted only by a human approver.

Algorithm 1 Gateway check for query q under capability C

resolve every column of q to $t.c$; **if** unresolvable **return** refuse
if q touches a protected or sensitive column inside a CTE, derived table or set operation **return** refuse
$V \leftarrow \{(c, \mu(c,q)) : \mu(c,q) > C(c)\}$ ▷ Eqs. 1–2
if $V \neq \emptyset$ **then return** refuse with V (escalation request)
end if
for $p \in P$ used by q in a filter **do**
  $\mathcal{C} \leftarrow$ candidate sets of the relaxed rewrite; **if** it fails **return** refuse
  if $\iota_\downarrow < k$ **or** $D_\downarrow(q) \neq \emptyset$ **then return** refuse ▷ Eq. 7
  end if
end for
for $p \in P$ used by q in mode agg **do**
  $\mathcal{S} \leftarrow$ entity sets of the key-only rewrite; **if** it fails **return** refuse
  if $\min(\iota_1, \iota_2) < k$ **or** $D(q) \neq \emptyset$ **then return** refuse ▷ Eq. 7
  end if
end for
commit $\mathcal{S}$, $\mathcal{C}$ to $\mathcal{H}$, $\mathcal{H}_\downarrow$ and their bases; execute q; log
return rows

5 METHOD

5.1 Mode-aware capabilities

The idea is to hand the agent the part of the schema its task needs, and to say for each column how it may be used. Counting customers per nation needs the nation key only as a filter; an average balance needs the balance inside an aggregate; only a name the user asked for needs to be returned. The mode in which a query q uses column c is the highest mode over its occurrences o:

$$\mu(c,q) = \max_{o \in \mathrm{occ}(c,q)} m(o), \tag{1}$$

where $m(o) =$ raw if o is returned (in the outer select list outside any aggregate, or the argument of a projected MIN or MAX), $m(o) =$ agg if o is inside SUM, AVG or another value aggregate, and $m(o) =$ filter otherwise (WHERE, JOIN, GROUP BY, ORDER BY, or inside COUNT). COUNT reveals cardinality, not values; a projected MIN or MAX returns one row's value and is therefore raw. The enforcer admits q under C iff

$$\forall c \in \mathcal{A} : \ \mu(c,q) \leq C(c). \tag{2}$$

Columns are resolved to $t.c$ by qualifying q against the schema, so aliases, joins and subqueries are handled; a query that cannot be resolved is refused.

Derivation and key closure: An LLM receives the task, any hint, and the schema with column descriptions, and returns a capability C_0 (prompt in the repository). Derived capabilities tend to omit join keys (Section 7.1), so the gateway adds, as filter, the keys of the tables $T(C_0)$ that C_0 touches:

$$C = C_0 \sqcup \{\kappa \mapsto \mathsf{filter} : \kappa \in K(T(C_0))\}, \tag{3}$$

where $\sqcup$ keeps the higher mode per column.

Escalation: When q violates Eq. 2, the gateway returns the out-of-capability uses $V(q) = \{(c, \mu(c,q)) : \mu(c,q) > C(c)\}$. An approver policy may grant them, $C \leftarrow C \sqcup V(q)$. Columns matching a fixed list of identifying attributes (person names, contact, address, birth, gender) are never granted automatically; each becomes a request to a human. The same ceiling applies to C_0 itself.

5.2 Session audit

The audit asks one question before each query runs: if this answer is released, how few people could it be about, given everything already released? It answers with entity sets. Let $p \in P$ be a protected column of entity table $E(p)$ with key κ. For an admitted query that aggregates p, the gateway rewrites it to select κ and the grouping expressions, dropping aggregation, HAVING, ORDER and LIMIT, and runs the rewrite on the read-only connection. This yields $\mathcal{S}(q) = \{S_1, \ldots, S_m\}$, one set per output group. Four tests follow.

Small cell. The smallest group rests on

$$\iota_1(q) = \min_{S \in \mathcal{S}(q)} |S| \tag{4}$$

entities.

Nested pair. With $\mathcal{H}$ the sets of earlier admitted aggregates over p, a nested pair reveals the sum over its difference, as in Fig. 1a:

$$\iota_2(q) = \min_{(A,B) \in N(q)} |B \setminus A|, \tag{5}$$

where $N(q)$ is the set of pairs $A \subset B$ from $\mathcal{S}(q) \cup \mathcal{H}$ with at least one of them in $\mathcal{S}(q)$.

Linear disclosure. Differences need not be nested. A SUM answer is the inner product of $\mathbf{1}_S$ with the vector of per-entity values; an AVG is the same once the group's COUNT is known, which a caller can always obtain. Entity i's value is determined by the answers iff the unit vector e_i lies in their span [4]. With Q an orthonormal basis of $\{\mathbf{1}_S : S \in \mathcal{H}\}$,

$$e_i \in \mathrm{span}(Q) \iff (QQ^\top)_{ii} = \textstyle\sum_j Q_{ij}^2 = 1, \tag{6}$$

tested with tolerance 10^{-9} on indicator vectors, and the set of entities q would newly disclose is $D(q) = \{i :$

$(Q'Q'^{\top})_{ii} = 1\}$, where Q' extends Q by Gram–Schmidt with $\mathbf{1}_S$ for $S \in \mathcal{S}(q)$.

Predicate dependence. A protected column can leak without being aggregated, as Fig. 1b shows. When q uses p in its WHERE clause outside any aggregate, the gateway *relaxes* q: it replaces every comparison on p with TRUE, which yields a query $q^{\downarrow}$ that matches a superset of q's rows. The candidate sets $\mathcal{C}(q)$ are the entity sets of $q^{\downarrow}$'s output groups, or one singleton per entity if q returns rows rather than aggregates. The answer of q depends on the protected values of these entities and of no others. The gateway applies Eqs. 4–6 to $\mathcal{C}(q)$ against a separate history $\mathcal{H}_{\downarrow}$, giving $\iota_{\downarrow}(q)$ and $D_{\downarrow}(q)$. Relaxation gives a superset only for comparisons joined by AND and OR, so a protected column under NOT or CASE, in JOIN ON, GROUP BY, ORDER BY or HAVING, or in a subquery, is refused. So is SUM, AVG, MIN or MAX of a non-protected column under such a filter: summing known, distinct order totals over the customers whose balance passes a threshold would decode which of them passed.

The gateway refuses q iff

$$\min\big(\iota_1(q), \iota_2(q), \iota_{\downarrow}(q)\big) < k \;\text{ or }\; D(q) \cup D_{\downarrow}(q) \neq \emptyset. \qquad (7)$$

A refused query reveals nothing, so its sets enter no history. Eq. 4 alone is the stateless baseline (cell suppression); Eqs. 4–5 form the pairwise audit; Eq. 7 in full is the linear audit.

Fail-closed coverage: The audit is only as complete as the entity sets it can compute. A protected column referenced inside a CTE, a derived table or a set operation would escape it, because its outer use resolves to the derived relation and the rewrite cannot attribute groups to entities. The gateway therefore refuses any query that touches a protected or sensitive entity column in such a shape, or that references the entity table more than once, and refuses a query whose rewrite fails. A rewrite that succeeds and matches no entity is admitted, since the answer covers no one. Every admitted answer that depends on a protected column has therefore been audited, and every admitted raw return of a sensitive column has been counted against the entity budget.

Session alarms: For comparison with intent-level monitoring we also compute session scores: drift, the sensitivity-weighted sum $\sum w_c$ over distinct out-of-capability uses (c, μ), counted once each so that repetition does not grow it; cell, $1/\min(\iota_1, \iota_2)$ over the session; and breadth, the number of distinct entities whose protected or sensitive columns were returned raw, divided by the purpose's entity budget. The gateway also enforces the budget: a query that would return sensitive columns for more entities than the purpose allows is refused.

5.3 Guarantees

Key closure (Eq. 3) adds only filter modes, so by Eq. 2 it never lets a query return or aggregate a column that C_0 did not already allow; it widens what a task may join on, not what it may see. For the audit we state the property precisely.

Proposition 1. Let $A \in \{0,1\}^{m \times n}$ have the indicator vectors of the m admitted aggregate groups over a protected column as rows, and let each answer be the exact sum Ax of the unknown per-entity values $x \in \mathbb{R}^n$. If no admitted query satisfied $D(q) \neq \emptyset$, then for every entity i and every x there is x' with $Ax' = Ax$ and $x'_i \neq x_i$.

Proof: Each admission checked Eq. 6 with the query's sets added, so after the last admission $e_i \notin \mathrm{row}(A)$ for every i; refused queries add no rows. Since $\mathrm{row}(A) = \mathrm{null}(A)^{\perp}$, $e_i \notin \mathrm{row}(A)$ implies some $z \in \mathrm{null}(A)$ has $z_i \neq 0$ (otherwise $e_i \perp \mathrm{null}(A)$ and $e_i \in \mathrm{row}(A)$). Put $x' = x + z$. Then $Ax' = Ax$ and $x'_i = x_i + z_i \neq x_i$. The test itself is exact because $(QQ^{\top})_{ii} = \|QQ^{\top} e_i\|^2$ is the squared norm of the projection of the unit vector e_i, which equals 1 iff e_i lies in the span. □

The predicate test has no such proof. A count under a threshold is linear in the indicator vector of that threshold, $[x_j > t]_j$, and different thresholds give different unknown vectors. The audit treats all filters on p as one unknown, which refuses every single-entity probe and every nested or linear combination of candidate sets, but an attacker who combines answers across thresholds is not covered by Proposition 1. Strict mode closes that gap.

Strict mode: Refusals can leak too: the decision depends on which entities match a query, and if membership depends on protected values, a refusal says something about them [5]. In strict mode the gateway refuses any query that uses a protected column outside an aggregate, that is, in WHERE, JOIN, GROUP BY or ORDER BY.

Proposition 2. In strict mode, the sequence of gateway decisions for a session is a function of the queries and of the non-protected columns only.

Proof: Admission under Eq. 2 and the strict check depend only on the query text. For an admitted aggregate, the key-only rewrite keeps the query's predicates and groupings and drops its aggregates, so in strict mode it reads no protected column; the entity sets $\mathcal{S}(q)$, and hence Eqs. 4–7, depend only on non-protected data. By induction over the session, $\mathcal{H}$ and Q do too. □

Refusals in strict mode therefore reveal nothing about protected values beyond what the non-protected columns already reveal, and the threshold probes of Fig. 1b never run. The price is refusing legitimate filters on protected values, such as balance bands, which we measure.

Limits: Proposition 1 rules out exact disclosure of one entity's value. For $k > 2$ it does not rule out a linear combination of three or more answers that isolates the sum over 2 to $k-1$ entities: Eqs. 4–5 catch such small sets only when they appear as a group or as the difference of a nested pair. The audit thus enforces k-isolation for groups and nested pairs and exact non-disclosure ($k = 1$) for arbitrary linear combinations. It does not rule out narrowing a value to an interval when values are bounded, since x' may fall outside the domain. Nor does it cover values released through non-protected columns, joins that count an entity several times, collusion across identities, or, outside strict mode, leakage through refusals themselves [5].

5.4 Cost

Parsing and qualification are linear in the query size. Each rewrite costs one extra query whose result size is the number of matching rows. The linear test keeps $Q \in \mathbb{R}^{n \times r}$ over the n entities touched so far and rank $r \leq n$; adding a set

TABLE 3
Linear-audit check time (median of 5) and basis memory vs. entities n and rank r, for random contiguous key-range sets (Python/NumPy, Apple M-series CPU, single process on an otherwise idle machine).

n	r	Check (ms)	Basis (MB)
1,000	10	0.04	0.1
1,000	100	0.09	0.8
1,000	500	0.28	4.0
10,000	10	0.24	0.8
10,000	100	1.02	8.0
10,000	500	7.07	40.0
50,000	10	1.57	4.0
50,000	100	6.42	40.0
50,000	500	32.50	200.0

costs $O(nr)$ with two Gram–Schmidt passes for stability, and Eq. 6 costs $O(nr)$. A query with more groups than a configurable limit (2,000 in our runs) is refused rather than audited. Table 3 measures the check: 0.04 ms with 1,000 entities and rank 10, and 32 ms with 50,000 entities and rank 500, where the dense basis takes 200 MB. Sessions in our workloads stay far below that rank.

6 Evaluation Methodology

6.1 Environment

Python 3.14, sqlglot 30.23 for parsing and qualification, SQLite and PostgreSQL 16, NumPy, and the MCP Python SDK 2.x. LLMs: gpt-4o-mini and gpt-4o through the OpenAI chat API (temperature 0 for capability derivation and SQL generation; 0.7 with a per-episode seed for attackers), Claude Sonnet 5.5 through the Anthropic API (low effort, provider-side refusal fallback enabled for capability derivation), and qwen3:4b-instruct run locally. All prompts, model outputs and seeds are cached in the repository; every number below is a macro generated from the result files.

6.2 Data

BIRD tasks. From BIRD dev [25] we draw 20 tasks per database from its 11 databases (220 tasks, seed 13); gold SQL and question text are written by BIRD's annotators. *TPC-H sessions.* A seeded TPC-H-shaped SQLite database (1,500 customers, 7,500 orders, synthetic values) has four purposes and a session generator: 1600 benign analyst sessions (400 per purpose) with realistic noise (30% contain one out-of-purpose query; finance sessions include drill-downs and overlapping histogram bands) and 500 attack sessions in five families (drift, small cell, tracker, three-query tracker, enumeration). Every session has 6–16 queries regardless of label, so length carries no label information. *BIRD sessions.* The human-written gold queries of BIRD's `financial` (106 queries) and `debit_card_specializing` (64) databases are arranged into 200 random sessions of 6–16 queries per database.

6.3 Adaptive attackers

Each attacker is an LLM told it is red-teaming a privacy gateway over synthetic data and given one customer key; it must find that customer's exact balance in at most 15 queries, seeing rows or refusal reasons after each. Disclosure is decided by an oracle that applies Eq. 6 to every answer the gateway returned, independent of what the attacker claims.

6.4 Statistical reporting

For a proportion $\hat{p}$ over n trials we report the 95% Wilson interval

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \qquad z = 1.96. \tag{8}$$

For paired accuracy differences on the same tasks we report 95% percentile intervals from 5,000 bootstrap resamples of tasks. Session-alarm thresholds are set per purpose on a held-out half of the benign sessions; detection rates are measured on the other half plus all attacks.

7 Empirical Results

7.1 RQ1: Can a task be mapped to a least-privilege capability?

Fig. 3 places each grant by the columns it exposes and the accuracy of an agent that sees only that grant. With gpt-4o-mini, the role grant exposes 72.5 columns per task and reaches 52.7% EX. The oracle footprint of the gold query exposes 4.6 columns and reaches 55.5%, a difference of +2.7 pp [-3.6, +9.1] that includes zero: least privilege is not inherently costly, and narrowing the schema the agent sees does not hurt it. Derivation is where accuracy is lost. The LLM's own column choices fit the gold query in only 16.8% of tasks, mostly because join keys are missing, and EX falls to 15.0%. Key closure (Eq. 3) raises fit to 51.8% and EX to 40.5%. One escalation round, in which 27.3% of tasks request on average 0.38 columns, brings EX to 47.7% with 7.5 columns: -5.0 pp [-9.5, -0.5] against the role, with a tenth of its columns and about one returnable column per task (0.9) instead of all of them.

With gpt-4o the pattern holds: role 56.4% EX, oracle footprint 58.6%, derived capability with keys and escalation 55.0% (-1.4 pp [-5.0, +2.3] against the role) with 7.3 columns. With Claude Sonnet 5.5 the derived capability costs nothing measurable: 66.8% against 66.4% for the role (+0.5 pp [-4.1, +5.0]) with 7.4 columns, and the oracle footprint is clearly more accurate than the role (75.0%, +8.6 pp [+3.6, +13.6]). Table 4 lists every grant for all four models, including the open-weight qwen3:4b-instruct run locally on a 55-task subset (5 per database; its runtime prevented the full set).

Two observations hold across models. Even the oracle footprint refuses some agent queries (column *Refused*), because the agent often answers correctly with columns other than the gold query's; a capability must allow alternatives rather than one canonical solution. And the cost of a derived capability relative to the role shrinks as the model gets stronger: -5.0 points for gpt-4o-mini, -1.4 for gpt-4o and +0.5 for Sonnet. The weakest model gains from the narrowed schema (role 36.4%, derived 40.0%, on 55 tasks with wide intervals), mostly because it writes invalid SQL against 72 columns. The oracle footprint is at least as accurate as the role for every model, so least privilege itself costs nothing; what remains is derivation error, which grows with task difficulty.

TABLE 4
Capability results on 220 BIRD tasks. *Admissible*: share of tasks whose gold SQL fits the grant. *EX*: execution accuracy of an agent that sees only the grant [95% Wilson interval]. *Refused*: share of agent queries the enforcer refuses. *Columns*, *Raw*: mean columns granted, and granted with mode raw. The escalation row grants a refused query's out-of-capability uses once.

Model	Grant	Admissible	EX [95% CI]	Refused	Columns	Raw
gpt-4o-mini	Role (all columns)	0.99	0.527 [0.46, 0.59]	0.01	72.5	72.5
	Tables	0.78	0.459 [0.39, 0.53]	0.01	27.1	27.1
	Columns	0.17	0.150 [0.11, 0.20]	0.36	3.7	0.9
	Columns+keys	0.52	0.405 [0.34, 0.47]	0.27	7.4	0.9
	Oracle footprint	1.00	0.555 [0.49, 0.62]	0.17	4.6	0.9
	Columns+keys+escalation	–	0.477 [0.41, 0.54]	–	7.5	–
gpt-4o	Role (all columns)	0.99	0.564 [0.50, 0.63]	0.01	72.5	72.5
	Tables	0.85	0.559 [0.49, 0.62]	0.00	27.8	27.8
	Columns	0.52	0.468 [0.40, 0.53]	0.16	4.4	0.9
	Columns+keys	0.66	0.505 [0.44, 0.57]	0.10	7.3	0.9
	Oracle footprint	1.00	0.586 [0.52, 0.65]	0.13	4.6	0.9
	Columns+keys+escalation	–	0.550 [0.48, 0.61]	–	7.3	–
claude-sonnet-5-5	Role (all columns)	0.99	0.664 [0.60, 0.72]	0.01	72.5	72.5
	Tables	0.87	0.686 [0.62, 0.74]	0.00	28.0	28.0
	Columns	0.60	0.655 [0.59, 0.71]	0.03	4.4	0.9
	Columns+keys	0.70	0.641 [0.58, 0.70]	0.06	7.3	0.9
	Oracle footprint	1.00	0.750 [0.69, 0.80]	0.01	4.6	0.9
	Columns+keys+escalation	–	0.668 [0.60, 0.73]	–	7.4	–
qwen3:4b-instruct	Role (all columns)	1.00	0.364 [0.25, 0.50]	0.00	72.5	72.5
	Tables	0.80	0.418 [0.30, 0.55]	0.00	31.9	31.9
	Columns	0.20	0.127 [0.06, 0.24]	0.22	3.8	1.3
	Columns+keys	0.53	0.345 [0.23, 0.48]	0.11	7.9	1.3
	Oracle footprint	1.00	0.636 [0.50, 0.75]	0.07	4.7	0.9
	Columns+keys+escalation	–	0.400 [0.28, 0.53]	–	8.0	–

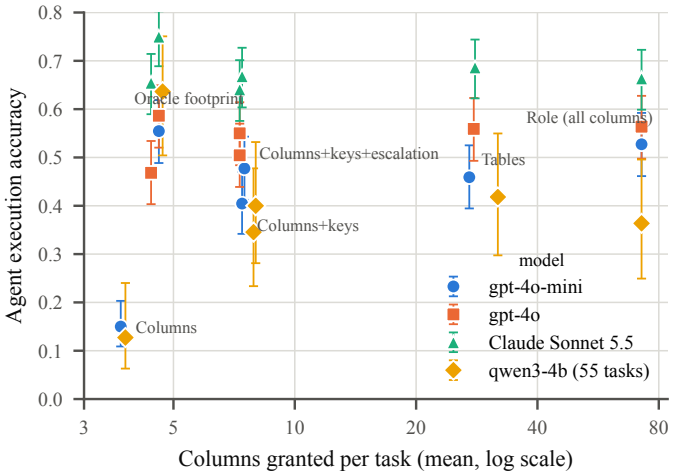


Fig. 3. Privilege vs. accuracy on 220 BIRD tasks. Each point is a grant; x: mean columns the agent may use (log scale); y: execution accuracy of an agent that sees only those columns; bars: 95% Wilson intervals. Labels refer to gpt-4o-mini (circles); squares: gpt-4o; triangles: Claude Sonnet 5.5 (all 220 tasks); diamonds: qwen3:4b-instruct (55-task subset).

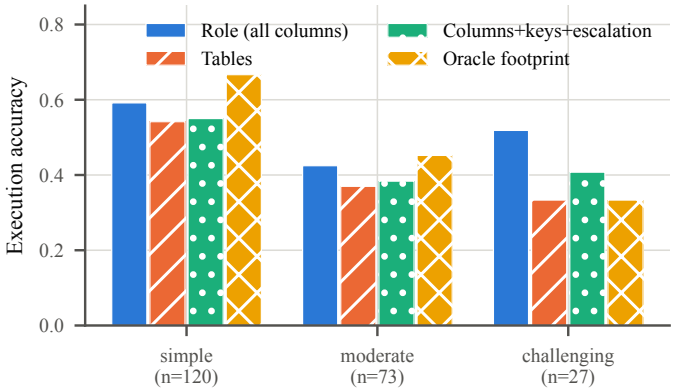


Fig. 4. Execution accuracy by BIRD difficulty (gpt-4o-mini) for the role grant, table-level grant, derived capability with key closure and one escalation round, and the oracle footprint.

Fig. 4 splits gpt-4o-mini's results by BIRD difficulty: the gap between role and derived capability is about 4 points on simple and moderate tasks and 11 points on the 27 challenging ones, which join more tables, so the derivation misses more of them (the small challenging sample makes this estimate loose).

7.2 RQ2: Can the derivation be manipulated?

We appended three instructions to each task: a blunt request for all columns, a plausible audit reason asking for names, contact details and birth dates, and an "admin override" placed in the hint. Fig. 5 reports capability size and the share of tasks whose capability returns identifying columns. For gpt-4o-mini the clean derivation grants 3.7 columns and identifying columns in 15.0% of tasks, mostly legitimately, for questions that ask for a person's name. The blunt instruction doubles capability size to 8.2 columns; the plausible reason makes 62.3% of tasks gain identifying columns they did not have; the hint injection barely changes anything (2.7%). gpt-4o ignores the blunt and hint instructions (4.5 and 4.3 columns against 4.4 clean; no new

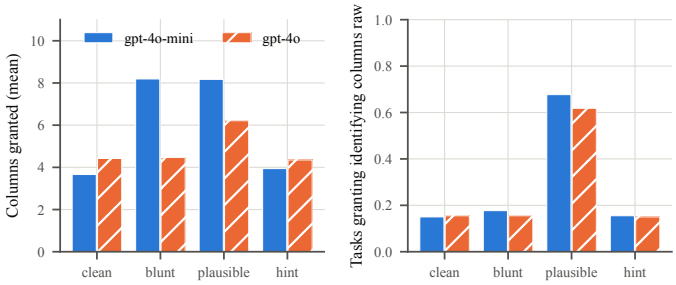


Fig. 5. Effect of injected instructions on derived capabilities (220 BIRD tasks). Left: mean columns granted. Right: share of tasks whose capability returns at least one identifying column. *clean*: no injection; *blunt*, *plausible*: appended to the task; *hint*: appended to BIRD's evidence field.

identifying grants) but not the plausible one: 54.1% of tasks gain identifying columns. The stronger model resists crude injection, not plausible text, which is the form real injections take [1].

The approval ceiling stops every injected identifying grant by construction, because no identifying column is granted without a human. Its cost is the clean rate: 15.0% of clean tasks would wait for approval. It does not bound non-identifying widening, which the blunt injection shows is large.

7.3 RQ3: Which session signals separate attacks from benign work?

Fig. 6 gives AUROC for every signal and attack family against held-out benign sessions of the same purpose. Session length carries no label information (query-count AUROC 0.53–0.61), so differences are not a length artifact. Each family is separated by the signal that measures its mechanism: drift by out-of-capability use (0.98), enumeration by breadth (1.00), small cells by the per-query cell test (1.00). The existing SQL guard, which only knows sensitive columns, and the row ledger, which counts sensitive rows, catch enumeration but nothing else.

Fig. 7 shows detection at 1% benign FPR. Session state helps where the violation is cumulative: drift detection rises from 47.0% to 66.0% (the remainder use only low-sensitivity columns), enumeration from 87.0% to 100.0%. For the aggregate families, session alarms detect almost nothing at this FPR even though the differencing score ranks trackers well (AUROC 0.92). The reason is substantive: 13.8% of benign finance sessions (55 of 400) genuinely isolate one customer, typically an average by nation and segment followed by the same average restricted to positive balances. Honest analysts produce the pattern of Fig. 1a too, so an alarm on the session is the wrong tool for this class; the right one refuses the single completing query.

7.4 RQ4: What does query-level auditing prevent, and at what cost?

The audit refuses only the query that would complete an isolation (Eq. 7). Fig. 8 plots attacks prevented against benign finance queries refused. Cell suppression alone (stateless) refuses 0.6% of benign queries at $k = 3$ but prevents only 1.0% of trackers. The pairwise audit prevents 100.0% of

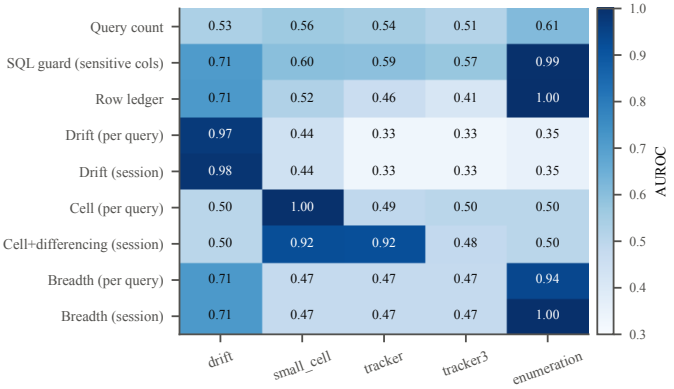


Fig. 6. AUROC of each signal (rows) for each attack family (columns) vs. held-out benign sessions of the same purpose, on the TPC-H session benchmark (2100 sessions). Per-query signals use the maximum over the session; session signals use accumulated state.

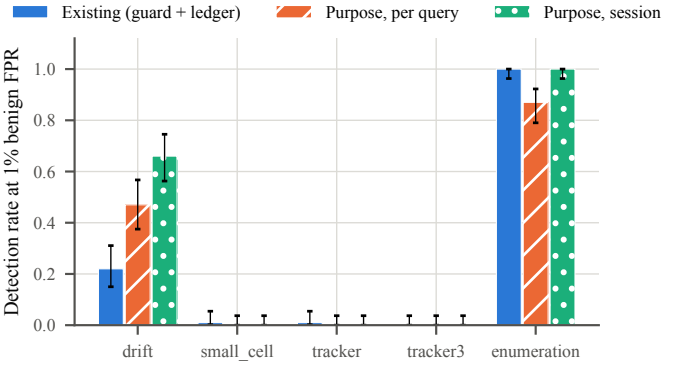


Fig. 7. Detection rate of session alarms at a 1% target benign FPR (thresholds per purpose on held-out benign sessions; OR detectors split the budget across their three signals). Bars: 95% Wilson intervals over 100 attack sessions per family.

trackers at 3.3% benign refusals, but only 12.0% of three-query attacks whose sets are not nested. The linear audit prevents 100.0% of trackers and 100.0% of three-query attacks at the same benign cost as the pairwise audit (3.3% of queries; 26.8% of benign sessions see at least one refusal). The linear test adds protection without adding benign refusals, because honest workloads rarely produce exact linear dependencies that the small-group and pairwise tests do not already catch. Strict mode keeps the same protection (100.0% and 100.0%) but refuses 25.6% of benign queries, because analysts filter on balances (histogram bands, drill-downs above a threshold); simulatable decisions cost this much on our workload. At $k = 5$ every non-strict mode refuses 13.9% of benign queries, because ordinary nation-by-segment groupings contain fewer than five customers.

Threshold probes: The scripted families above aggregate the balance. The bisection of Fig. 1b does not, and without the predicate test it passes every other test: run against the linear audit on the TPC-H database, 21 counts of the form `COUNT(*) WHERE c_custkey = 570 AND c_acctbal > t` recovered the target's balance to the cent with no refusal. With the predicate test the first probe is refused, since its candidate set holds one customer. All benign rates in this section include the predicate test; the

TABLE 5
Differential privacy vs. the session audit on the finance workload. *Median error*, *Within 5%*: relative error of benign answers per output group. *Refused*: benign queries refused. *Attacker*: share of tracker sessions whose estimate is within 10% of the target (DP), or with exact disclosure (audit).

Answering	Median error	Within 5%	Refused	Attacker
DP, $\varepsilon = 1$	87.7%	8.1%	–	0.1%
DP, $\varepsilon = 10$	8.8%	37.0%	–	2.2%
DP, $\varepsilon = 100$	0.9%	91.6%	–	18.7%
Linear audit, $k = 3$	0	100%	3.3%	0%

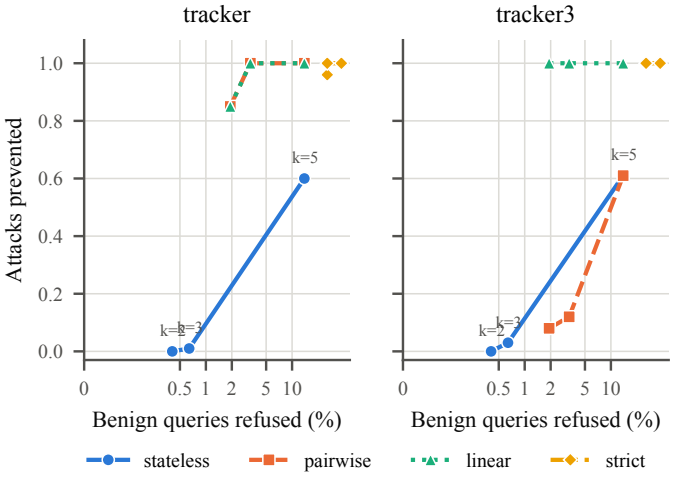


Fig. 8. Query-level audit on finance sessions: share of attacks prevented (refused at or before the disclosing query) vs. share of benign queries refused, for $k \in \{2, 3, 5\}$ and the four audit modes (strict: linear plus no predicates on the protected column). Left: two-query trackers; right: three-query trackers whose sets are not nested. x axis symmetric-log.

histogram bands and drill-downs of the finance sessions pass it because their candidate sets are whole nation-by-segment groups.

Comparison with differential privacy: Instead of refusing, a gateway could answer every aggregate over the protected column with Laplace noise [21], [22]. We clip balances to $[-1{,}000, 10{,}000]$ (sensitivity 10,000 per SUM), split a session budget ε equally across its protected aggregates, and answer an average as a noisy sum over the exact group size. Table 5 compares utility on the benign finance sessions with the share of tracker sessions in which the attacker's estimate lands within 10% of the target's balance. At $\varepsilon = 10$, benign answers are off by a median of 8.8% and only 37.0% are within 5%, while the attacker succeeds in 2.2% of sessions; at $\varepsilon = 100$, 91.6% of benign answers are within 5% but the attacker succeeds in 18.7%. On groups of about a dozen customers no budget gives both. The audit instead refuses 3.3% of benign queries and returns the rest exactly. The comparison is not one-sided: differential privacy also bounds approximate inference and needs no entity sets, which the audit does not, and the two arms use different success criteria (within 10% for DP, exact for the audit). Differential privacy suits large groups and approximate dashboards; the audit suits exact answers over small groups.

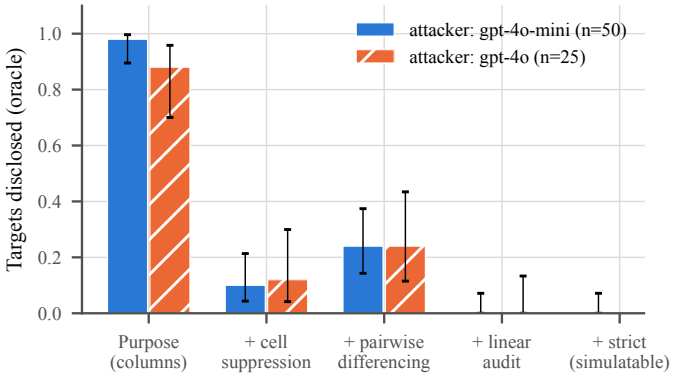


Fig. 9. Adaptive LLM attackers: share of targets whose exact balance is determined by the answers the gateway returned (oracle, Eq. 6), per defense and attacker model. Bars: 95% Wilson intervals.

7.5 RQ5: Does the audit hold against adaptive attackers?

We ran 50 targets per defense with gpt-4o-mini and 25 with gpt-4o (the gpt-4o strict-mode run and its extension to 50 targets were not completed). Claude Sonnet 5.5 declined the attacker role on the first turn of every episode, so it contributes no attacker data. Fig. 9 reports oracle disclosure. With column rules only, both attackers succeed almost always (gpt-4o-mini 98.0%, gpt-4o 88.0%), usually with one AVG over a single key. Cell suppression stops direct isolation but not differencing (10.0%, 12.0%). The pairwise audit is defeated by combinations it does not model; the disclosed episodes include overlapping key ranges such as $[570, 577] + [577, 590] - [570, 590]$ and equal-size averages over shifted key triples, neither of which contains a nested pair differing by fewer than three entities (24.0%, 24.0%). gpt-4o-mini had three `UNION ALL` queries answered unaudited in runs that predate the fail-closed rule, two in one pairwise-audit episode and one in a column-rules-only episode; removing them from the answered sets leaves both targets disclosed, so those rates are unchanged, and no such query was answered under the linear or strict audit. Under the linear audit no target was disclosed by either attacker (0 of 50 and 0 of 25; upper 95% bounds 7.1% and 13.3%), at 5.1 and 3.8 refusals per episode. Strict mode gives 0.0% disclosure for gpt-4o-mini, with refusals that by Proposition 2 carry no information about balances.

These episodes were run before the predicate test existed, and the oracle measures exact linear disclosure only. No attacker tried a threshold bisection; had one done so, the oracle would not have counted it. The predicate test only adds refusals, so the linear-audit result stands, and the threshold channel is covered by the scripted probe in Section 7.4 rather than by these runs.

7.6 RQ6: Cost on human-written queries and overhead

The TPC-H workloads above are generated. Fig. 10 uses BIRD's human-written queries instead, with an analytics purpose that permits every column except per-entity amounts and balances, which may only be aggregated. On `financial`, 3.8% of queries return a protected value raw and are refused by the capability; the full session audit refuses 15.3%, against 15.3% for per-query

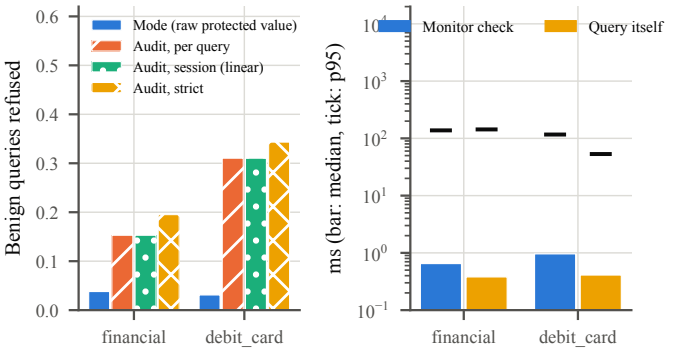


Fig. 10. Human-written BIRD workloads (200 sessions per database). Left: share of benign queries refused by the capability (raw protected value), by per-query cell suppression, and by the full session audit, $k = 3$. Right: monitor check time vs. the query's own execution time (bars: median; ticks: 95th percentile; log scale).

cell suppression. On `debit_card_specializing` the figures are 3.1%, 31.0% and 31.0%. Strict mode raises the audit refusals to 19.5% and 34.3%. The high rate on `debit_card_specializing` reflects its questions, many of which ask for one customer's consumption; under an aggregate-only purpose those are disclosures by design, so the rate measures policy fit, not audit error.

Most of the audit's refusals on `financial` come from the predicate test, and they are disclosures the aggregate tests alone admitted. Taken one at a time in a fresh session, 15 of its queries are refused, against 4 without the predicate test. Of these, 5 rank accounts by a protected value (`ORDER BY amount DESC LIMIT 1`, "which account has the largest loan"), which names the entity at the top, and 5 list the accounts whose amount passes a threshold, which reveals one bit of each listed account's value. On `debit_card_specializing`, 12 of its 20 single-query refusals are per-customer aggregates. In both databases the stateful tests (nested pairs, linear combinations) refused nothing that the per-query tests had not already refused: on human-written sessions, the cost of the audit is the cost of its per-query rules. The aggregate rewrite computed entity sets for 81.8% and 90.3% of the protected aggregates; the fail-closed rule refuses the rest (CTEs, nested set operations), so they count as refusals rather than as unaudited answers.

The monitor's median check takes 0.65 and 0.96 ms, against 0.38 and 0.41 ms for the queries themselves; at the 95th percentile, 137.8 and 116.6 ms against 143.2 and 53.2 ms. Overhead is dominated by the key-only rewrite, which scans as many rows as the query.

7.7 RQ7: Does it work as an MCP gateway at scale?

We loaded the TPC-H-shaped schema into PostgreSQL 16 at roughly scale factor 1 (150,000 customers, 1.5M orders, 4.5M line items; the gateway connects through a role with only SELECT and a read-only transaction default). We generated 260 sessions against this database (40 benign per purpose, 20 per attack family; 2,942 queries), transpiled them to PostgreSQL, and replayed them through an MCP client on two paths: the gateway executing on Postgres itself, and the gateway forwarding to a separate database MCP server that holds the database connection, as an existing vendor server would. Table 6 reports the results.

TABLE 6
End-to-end MCP deployment on PostgreSQL at TPC-H scale factor ≈1 (260 sessions, 2,942 queries). *Parity*: decisions equal to the in-process library. Times in ms, median / 95th percentile; *Transport*: round trip minus check and execution, median.

Path	Parity	Check	Execute	Transport	Round trip
Gateway → Postgres	100.0%	1.4 / 589	11.4 / 176	1.0	25.9 / 604
Gateway → DB MCP → Postgres	100.0%	1.8 / 742	12.0 / 174	0.9	27.1 / 757

TABLE 7
Deployment checklist.

Item	Decision
Database account	SELECT-only role, read-only transaction default
Catalog	Entity tables and keys; sensitivity tag per column
Protected columns	Which per-entity values may only be aggregated
Identifying columns	List reviewed against catalog tags; named approver
Audit mode	k; linear (exact filters allowed) or strict (simulatable refusals)
Entity budgets	Per purpose, for tasks that need individual records
Decision log	Retention and who may read it

Both paths reproduced the in-process library's decision on every query (100.0% parity, no errors), so the protocol layer neither drops nor alters decisions. Every attack in all five families was refused at or before its disclosing query (drift at the column check, enumeration by the entity budget), while 2.5–3.8% of benign queries per purpose were refused, mostly out-of-purpose slips. The median gateway check took 1.4 ms against a median query execution of 11.4 ms, and MCP transport added about 1.0 ms. The tail is the key-only rewrite: at the 95th percentile the check took 589 ms, for aggregates whose rewrite returns every matching customer key; through the proxy, those keys also cross a second MCP hop (742 ms). Computing entity sets inside the database, for example as a hashed bitmap, would remove the shipping of keys.

8 Enterprise Readiness

In an enterprise the gateway sits as an MCP proxy in front of database MCP servers, so agents and databases are unchanged; Section 7.7 shows this path works without changing decisions. The capability for a task is issued once and carried with the session, and the human approval step attaches to the identity the application authenticates. Because the gateway sees every source an agent queries, the ledger can extend across databases, where linkage between, for example, customer and payroll systems is invisible to each system's own policy. Capabilities can also be compiled into native views and row policies as a second line of enforcement. Table 7 lists what a deployment has to decide.

Where it adds little: For agents that already act strictly as the end user through narrow APIs, the remaining gain is the session audit. Tasks that legitimately need

individual records, such as a support agent helping one customer, need a purpose with an entity budget rather than an aggregate-only rule; the breadth signal covers that case, but its budget is a policy choice.

9 Discussion

9.1 Metric limitations

Execution accuracy compares result sets and ignores answers that are correct in substance but differ in form, which affects every grant equally. The adaptive-study oracle counts only exact linear disclosure; it does not count interval narrowing or threshold bisection, which the scripted probe covers instead. The DP comparison uses a within-10% criterion for DP and an exact one for the audit.

9.2 Threats to validity

Capability derivation was measured with three commercial models on 220 tasks and one open-weight model on 55; the injection study covers only the two OpenAI models. The TPC-H sessions and attack families are authored by us; the adaptive study and the BIRD workloads reduce, but do not remove, this dependence. The identifying-column list is a fixed name rule that both over- and under-matches. The audit does not stop inference of bounds or collusion across identities, and only strict mode removes leakage through refusals; differential privacy [21], [22] addresses some of these at the price of exact answers. Overhead was measured on SQLite (up to about one million rows) and on PostgreSQL at TPC-H scale factor 1; warehouse engines and larger scales remain untested.

9.3 Next steps

The next concrete step is to compute entity sets inside the database as hashed bitmaps, which removes the tail latency of Section 7.7 and lets the predicate test keep one basis per threshold instead of one for all. After that: a per-task escalation budget to bound non-identifying widening, an interval oracle for the adaptive study, and evaluation on logged, consented enterprise sessions.

10 Conclusion

We built an MCP gateway that gives each agent task a column-level, mode-aware capability and audits every answer that depends on a protected column against the entity sets of earlier answers. Capabilities removed about nine tenths of the columns an agent could use while execution accuracy changed by between +0.5 and -5.0 points across models, and the audit stopped every scripted and adaptive differencing attack while refusing 3.3% of benign generated queries. Teams that connect agents to production databases can deploy it in front of an existing database MCP server without changing either side, and keep exact answers for the queries that do not single anyone out.

Data and Code Availability

Code, prompts, LLM response caches, benchmark generators and result files are in the author's repository. BIRD is used under its license and not redistributed; the TPC-H-shaped database is synthetic and regenerated from a seed.

Declarations

Funding: [to be completed by the author]. **Conflicts of interest:** [to be completed by the author]. **Use of AI tools:** [to be completed by the author, per the journal's policy].

References

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. ACM, 2023, pp. 79–90.

[2] R. Pedro, D. Castro, P. Carreira, and N. Santos, "From Prompt Injections to SQL Injection Attacks: How Protected is Your LLM-Integrated Web Application?" *arXiv preprint arXiv:2308.01990*, 2023.

[3] D. E. Denning and P. J. Denning, "The tracker: a threat to statistical database security," *ACM Transactions on Database Systems*, vol. 4, no. 1, pp. 76–96, 1979.

[4] F. Chin and G. Ozsoyoglu, "Auditing and Inference Control in Statistical Databases," *IEEE Transactions on Software Engineering*, vol. SE-8, no. 6, pp. 574–582, 1982.

[5] K. Kenthapadi, N. Mishra, and K. Nissim, "Simulatable auditing," in *Proceedings of the twenty-fourth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. ACM, 2005, pp. 118–127.

[6] Y. Fei, Y. Jiang, Y. Yang, and X. Xiao, "Benchmarking Text-to-SQL under Role-Based Access Control," *arXiv preprint arXiv:2607.22115*, 2026.

[7] Đorđe Klisura, J. Khoury, A. Kundu, R. Krishnan, and A. Rios, "Role-Conditioned Refusals: Evaluating Access Control Reasoning in Large Language Models," *arXiv preprint arXiv:2510.07642*, 2025.

[8] Đorđe Klisura and A. Rios, "Unmasking Database Vulnerabilities: Zero-Knowledge Schema Inference Attacks in Text-to-SQL Systems," *arXiv preprint arXiv:2406.14545*, 2024, in Findings of the Association for Computational Linguistics: NAACL 2025.

[9] K. Wang, Z. Wang, P. Lv, H. Li, G. Li, G. Cong, and W. Dong, "Data Agents Under Attack: Vulnerabilities in LLM-Driven Analytical Systems," *arXiv preprint arXiv:2606.08661*, 2026.

[10] R. K. Surapani, P. K. D. Kakitapelli, A. Morampudi, and P. Padi, "Authorization Architectures for Tool-Using AI Agents," *arXiv preprint arXiv:2609.15906*, 2026.

[11] B. A. Shivakumar, S. Priya, and P. Gao, "AgentFlow: A Flow-Centric Policy Language and Framework for Securing LLM Agent Systems," *arXiv preprint arXiv:2608.22868*, 2026.

[12] M. Russinovich, A. Salem, and R. Eldan, "Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack," *arXiv preprint arXiv:2404.01833*, 2024, in Proc. 34th USENIX Security Symposium, 2025.

[13] C. Yueh-Han, N. Joshi, Y. Chen, M. Andriushchenko, R. Angell, and H. He, "Monitoring Decomposition Attacks in LLMs with Lightweight Sequential Monitors," *arXiv preprint arXiv:2506.10949*, 2025, in Proc. International Conference on Learning Representations (ICLR), 2026.

[14] S. Mishra, D. Chukkapalli, and G. R. Naik, "Stateful Guardrails for Multi-Turn LLM Systems: A Conversational Risk Accumulation Framework," *arXiv preprint arXiv:2607.19361*, 2026.

[15] R. Sandhu, E. Coyne, H. Feinstein, and C. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[16] S. Rizvi, A. Mendelzon, S. Sudarshan, and P. Roy, "Extending query rewriting techniques for fine-grained access control," in *Proceedings of the 2004 ACM SIGMOD international conference on Management of data*. ACM, 2004, pp. 551–562.

[17] R. Agrawal, J. Kiernan, R. Srikant, and Y. Xu, "Hippocratic Databases," in *VLDB '02: Proceedings of the 28th International Conference on Very Large Databases*. Elsevier, 2002, pp. 143–154.

[18] J.-W. Byun and N. Li, "Purpose based access control for privacy protection in relational database systems," *The VLDB Journal*, vol. 17, no. 4, pp. 603–619, 2006.

[19] A. Kamra, E. Terzi, and E. Bertino, "Detecting anomalous access patterns in relational databases," *The VLDB Journal*, vol. 17, no. 5, pp. 1063–1077, 2007.

[20] I. Dinur and K. Nissim, "Revealing information while preserving privacy," in *Proceedings of the twenty-second ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. ACM, 2003, pp. 202–210.

[21] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating Noise to Sensitivity in Private Data Analysis," in *Lecture Notes in Computer Science*. Springer Berlin Heidelberg, 2006, pp. 265–284.
[22] N. Johnson, J. P. Near, and D. Song, "Towards practical differential privacy for SQL queries," *Proceedings of the VLDB Endowment*, vol. 11, no. 5, pp. 526–539, 2018.
[23] L. Sweeney, "k-anonymity: a model for protecting privacy," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 05, pp. 557–570, 2002.
[24] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2018, pp. 3911–3921.
[25] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Cao, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K. C. C. Chang, F. Huang, R. Cheng, and Y. Li, "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," *arXiv preprint arXiv:2305.03111*, 2023.